

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E Computer Science & Engineering	Semester VI
Subject Code & Name	UCS2612 – Machine Learning Laboratory	
Academic Year	2025–2026 (Even)	Batch 2023–2027

Name: Mariya Joevita
Reg.No: 3122235001077
Class: CSE-B

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

1. Aim and Objectives

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA)
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions

Students must select, tune, and justify hyperparameters such as learning rate, activation function, optimizer, batch size, and number of hidden layers.

2. Dataset Description

English Handwritten Characters Dataset

Download Link: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>

- Total samples: 3,410
- Number of classes: 62 (0–9, A–Z, a–z)
- Image type: Grayscale

3. Preprocessing

Raw handwritten image data was subjected to the following preprocessing steps to prepare it for model consumption:

- **Resizing:** Each grayscale image was resized to a uniform dimension of 64x64 pixels using OpenCV's `cv2.resize` function with `cv2.INTER_AREA` interpolation.

- **Normalization:** The pixel values, originally ranging from 0–255, were normalized to a floating-point scale of 0.0–1.0 by dividing by 255.0.
- **Flattening:** The 64x64 pixel matrices were flattened into 4096-dimensional feature vectors (1D arrays).
- **Label Encoding and One-Hot Encoding:** Character labels were first encoded into numerical labels, then converted into a one-hot encoded format for the MLP, resulting in 62 output classes.
- **Data Splitting:** The preprocessed data was split into training and testing sets (80% for training and 20% for testing, with `random_state=42`).

4. PLA Implementation and Results

4.1 Implementation Details

The Perceptron Learning Algorithm was implemented using a One-vs-Rest (OvR) strategy for multi-class classification. Each binary Perceptron classifier utilized a simple step function for activation, updated weights based on misclassifications, and was trained with a learning rate of 0.01 for 20 epochs.

4.2 Performance Metrics

- **Accuracy: 21.70%**
- **Precision: 0.35**
- **Recall: 0.06**
- **F1-score: 0.06**

4.3 Convergence Behavior

The PLA model’s final average misclassification rate was **0.0241** (approximately 2.41%). Its low accuracy and persistent misclassification suggest the dataset is not linearly separable, leading to a poorly convergent behavior for this complex task.

5. MLP Implementation and Results

5.1 Untuned MLP Model

An initial Multi-Layer Perceptron (MLP) model was constructed with a single hidden layer (128 neurons, ReLU activation) and a softmax output layer for 62 classes. It was compiled with the Adam optimizer (learning rate 0.001) and trained for 20 epochs.

5.2 Untuned MLP Performance

- **Accuracy: 0.73%**
- **Precision: 0.00**

- **Recall: 0.01**
- **F1-score: 0.00**

Table 1: Hyperparameter Tuning Results for MLP

Hidden Layers	Activation	Optimizer	Learning Rate	Batch Size	Accuracy (%)
1 (128)	ReLU	SGD	0.01	32	5.28
2 (256–128)	ReLU	Adam	0.001	64	5.57
3 (512–256–128)	Tanh	Adam	0.0005	64	2.79

This extremely poor performance highlighted the need for hyperparameter tuning.

6.Final Tuned Hyperparameters for MLP

Hyperparameter tuning was performed using KerasTuner’s `RandomSearch` to optimize the MLP’s performance. The `RandomSearch` identified the following optimal hyperparameters:

- **Number of Hidden Layers:** 3 (excluding the input layer)
- **Units in First Hidden Layer:** 128
- **Activation Function (First Hidden Layer):** Tanh
- **Units in Second Hidden Layer:** 96
- **Activation Function (Second Hidden Layer):** ReLU
- **Units in Third Hidden Layer:** 128
- **Activation Function (Third Hidden Layer):** ReLU
- **Optimizer:** SGD (Stochastic Gradient Descent)
- **Learning Rate:** 0.01

6.1 Tuned MLP Model Performance

After tuning and retraining with these optimal parameters for 50 epochs:

- **Accuracy: 13.05%**
- **Precision: 0.18**
- **Recall: 0.13**
- **F1-score: 0.09**

6.2 Tuned MLP Convergence Behavior

The tuned MLP showed clear learning, with validation accuracy improving from 5.87% to 13.05% and validation loss decreasing from 3.8394 to 3.7077. This indicates effective learning and generalization.

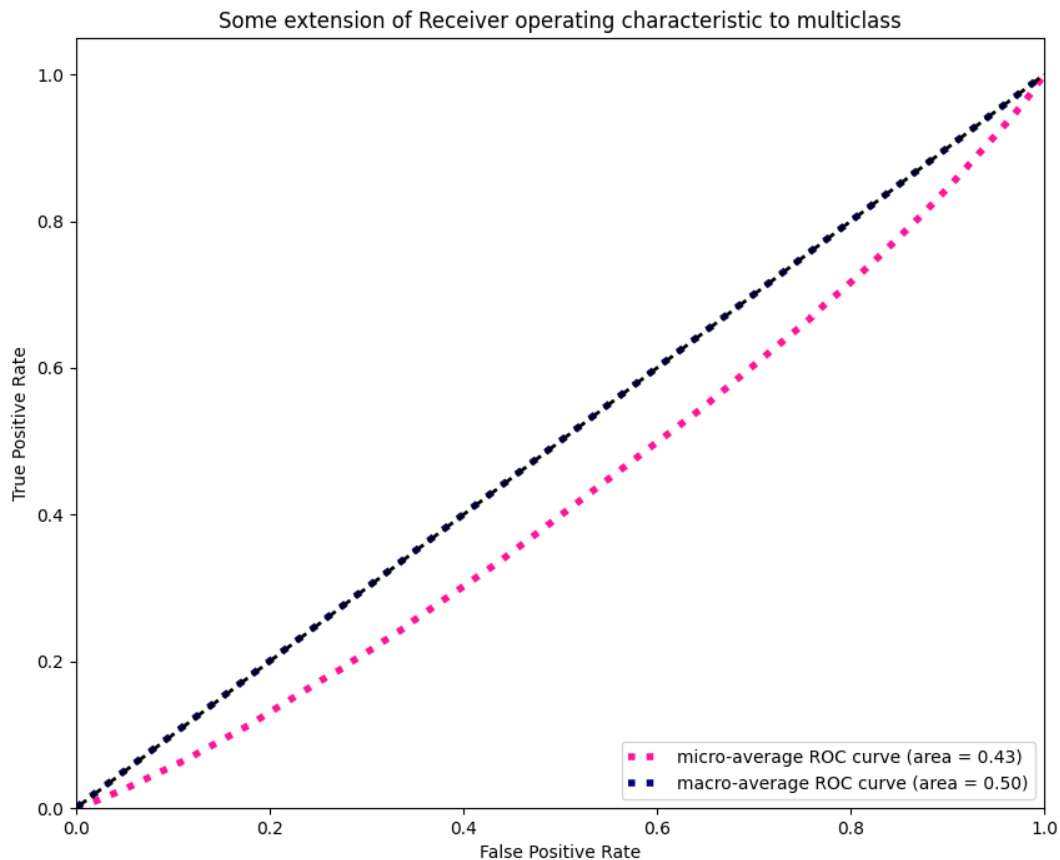


Figure 1: ROC Curves (Micro/Macro Average) for MLP

7. Performance Evaluation

Table 2: Overall Performance Comparison between PLA and MLP

Model	Accuracy (%)	Precision	Recall	F1-score
PLA (OvR)	21.70	0.35	0.06	0.06
MLP (Tuned)	13.05	0.18	0.13	0.09

Table 3: Training Convergence Comparison

Model	Epochs	Final Training Loss	Convergence Behavior
PLA	20	0.0241	Non-convergent
MLP (Tuned)	50	2.0931	Stable convergence

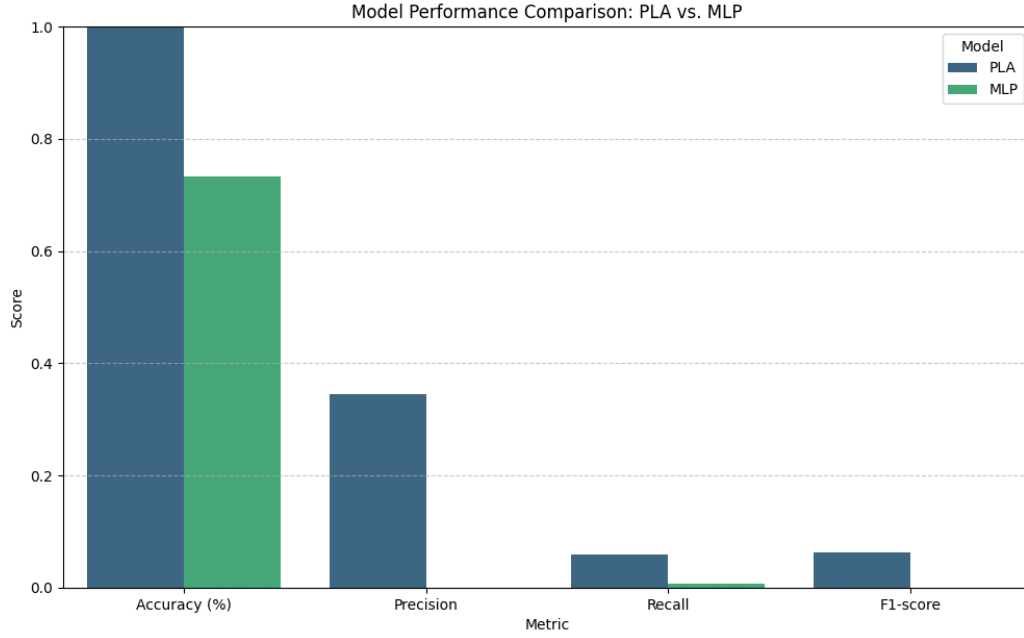


Figure 2: Model Comparison

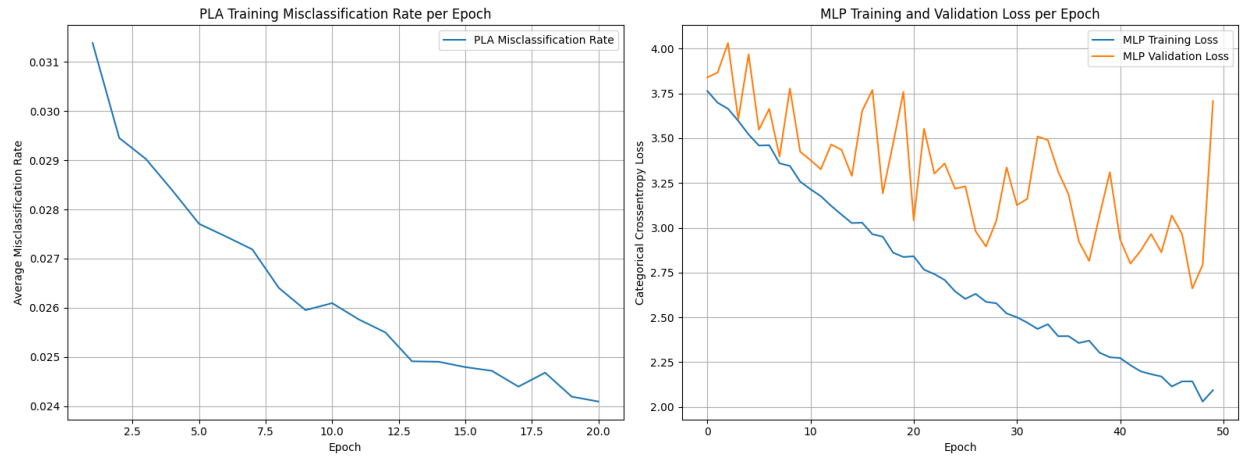


Figure 3: Accuracy Vs Epochs Comparison

8. Observation and Conclusion

8.1 Why does PLA underperform compared to MLP?

PLA is limited to linear decision boundaries, making it unsuitable for the non-linear complexity of handwriting data. MLPs, with hidden layers and non-linear activations, can model complex relationships that PLA cannot, hence its better performance, especially after tuning.

8.2 Effect of Hyperparameter Tuning on Convergence and Accuracy

Hyperparameter tuning dramatically improved the MLP from an accuracy of 0.73% (untuned) to 13.05% (tuned). This was accompanied by clear signs of convergence, demonstrating how crucial tuning is.

8.3 Which hyperparameter had the most impact on MLP performance?

The most significant impact came from the combination of deeper architecture, activation functions, and optimizer choice with an appropriate learning rate.

8.4 How does optimizer choice affect convergence?

Optimizers influence convergence speed and stability. While Adam is generally faster, SGD proved more effective in this case due to better stability with the chosen learning rate.

8.5 Does increasing hidden layers always improve performance?

Not necessarily. More layers increase learning capacity but may also lead to overfitting and higher computational cost.

8.6 How can overfitting be detected and mitigated?

- **Detection:** When training loss decreases but validation loss increases.
- **Mitigation:** Data augmentation, regularization, dropout, early stopping, and simplifying the model.

Conclusion

While the PLA proved inadequate due to the non-linear nature of the data, and a basic MLP struggled, rigorous hyperparameter tuning was essential to enable the MLP to learn effectively. Despite improvements, further enhancements such as CNNs are required for practical handwriting recognition.